Shared Space App: Use Cases

Nov 6, 2026 · @s1lent_of

1. Product Snapshot

- **Problem:** *students, who live on campus in a shared apartment, always forget to take out the rubbish, buy the washing liquid, clean the shared space weekly and a lot of other different things that has to be done in turns;*
- **Target users:** *students; people, who share room;*
- **Core value:** *our main goal is to simplify students routine by tracking who did what so household chore is shared fairly;*
- **Out of scope for the hackathon:** *admin panel, accommodation offices integration, different hardware sensors, trained and polished AI agent;*

2. Actors

- **Resident:** *any member of the room who uses the app;*
- **System:** automatic actions such as reminders and notifications;
- **AI Bot/Agent:** *simple AI integration through external APIs, suggesting who should do the next chore, based on history;*

3. Use Case Index

- UC-01 [MVP]: *Create a room for specific apartment and join it by QR*
- UC-02 [MVP]: *Assign and complete a chore*
- UC-03 [MVP]: *Shared shopping list*

4. Use Cases

UC-01: Create a room and join it by QR code

- **Priority:** MVP
- **Primary actor:** Resident
- **Goal:** Get everyone in the apartment into one shared room with no sign-up friction.
- **Trigger:** A student moves into a shared apartment, or a new flatmate arrives.
- **Preconditions:** The app is installed; camera access is allowed for scanning.

Main flow

1. Resident A taps "Create room" and enters a room name (e.g. "Flat 12B") and their own display name.
2. System creates the room and shows a QR code (plus a short text code as backup).
3. Resident B opens the app and taps "Join room".
4. Resident B scans the QR code and enters their display name.
5. System adds Resident B to the room and opens the room home screen.
6. Both residents see each other in the member list.

Alternative / error flows

- 4a. Camera access is denied or the scan fails: Resident B types the short code instead.
- 4b. The code is invalid or the room no longer exists: system shows an error and lets B retry.
- 4c. B is already a member: system opens the room without adding a duplicate.
- 1a. Resident leaves the name empty: system asks for a name before continuing.

Postconditions: The room exists with 2 or more members, and all members see the same room data.

Acceptance check: Create a room on phone A, scan its QR on phone B, and both phones list both names within 10 seconds.

UC-02: Assign and complete a chore

- **Priority:** MVP
- **Primary actor:** Resident
- **Supporting actors:** System (reminders, history), AI Bot/Agent (suggests who is next)
- **Goal:** Every chore, such as taking out the rubbish or weekly cleaning, has an owner and is shared fairly.
- **Trigger:** A chore needs to be done, or a recurring chore is due.
- **Preconditions:** The resident is in a room with at least two members (UC-01).

Main flow

1. Resident taps "Add chore" and enters a title (e.g. "Take out the rubbish"), and optionally a repeat interval (e.g. weekly) and whether it is for one person or the whole room.
2. System asks the AI agent for a suggestion of who should do it next, based on the history of completed chores.
3. AI agent returns a suggested assignee with a short reason (e.g. "Anna did the fewest chores this month").

4. Resident accepts the suggestion; system assigns the chore.
5. Assigned resident gets a reminder when the chore is due.
6. Assigned resident marks the chore as done.
7. System logs who did it and when, notifies the other residents, and schedules the next occurrence for the next person.

Alternative / error flows

- 4a. Resident picks someone else manually: system assigns the chore to that person instead.
- 3a. The AI service is unavailable or times out: system falls back to simple rotation between members.
- 5a. The chore is overdue: system re-sends the reminder and shows the chore highlighted to the whole room.
- 6a. Assignee cannot do it: they tap "Pass" and the chore goes to the next suggested person.
- 1a. A chore is general (for the whole room): any resident can mark it done, and the person who did it is recorded.

Postconditions: The chore is logged as completed with the person's name and time; the next occurrence is scheduled; the history used for suggestions is updated.

Acceptance check: Add a chore on phone A; phone B shows who it is assigned to and why. Mark it done on that phone, and phone A shows it as completed.

UC-03: Use the shared shopping list

- **Priority:** MVP
- **Primary actor:** Resident
- **Supporting actors:** System (live updates, notifications)
- **Goal:** Everyone can see what the household is running out of and avoid buying duplicates or forgetting essentials (e.g. washing liquid).
- **Trigger:** A resident notices something is running low, or someone is about to go shopping.
- **Preconditions:** The resident is in a room (UC-01).

Main flow

1. Resident opens the shopping list and taps "Add item".
2. Resident enters the item name (and optionally a quantity).
3. System adds the item to the list and shows who added it.
4. Other residents see the new item on their phones.

5. While shopping, a resident ticks off the items they bought.
6. System marks those items as bought, with the buyer's name and time, and moves them to a "Bought" section.

Alternative / error flows

- 2a. The item already exists on the list: system warns and offers to increase the quantity instead of adding a duplicate.
- 5a. A resident removes an item added by mistake: system deletes it for everyone.
- 5b. No internet connection: the change is kept on the device and synced when the connection returns.
- 6a. A resident un-ticks an item bought by mistake: it returns to the active list.

Postconditions: The list is identical on all members' phones; bought items are recorded.

Acceptance check: Add "Washing liquid" on phone A; it appears on phone B within 10 seconds, and ticking it on B shows it as bought on A.

7. Open Questions

- *What do we still need to decide before building?*
- *Which use case is the demo story for the final pitch?*